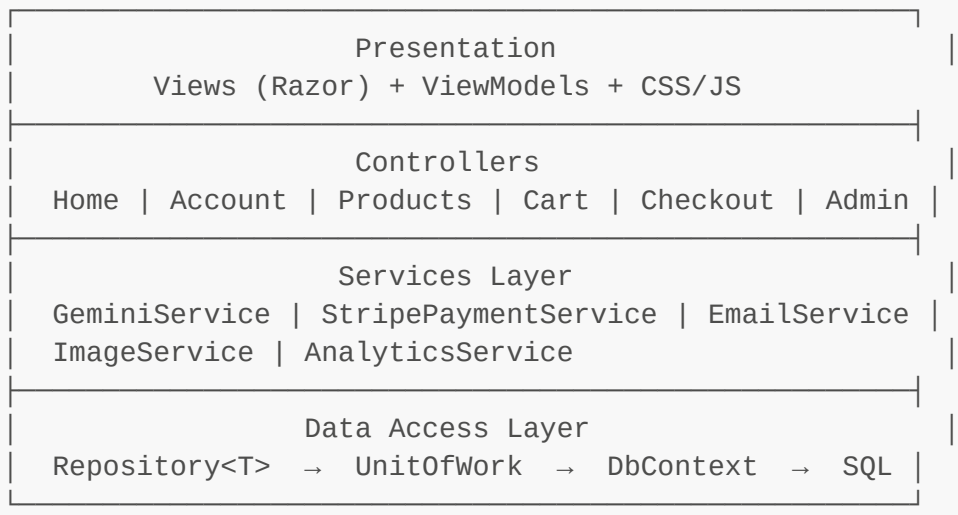
Sec-5: Backend Architecture — ASP.NET Core MVC E-Commerce Engine

5.1 Project Overview and Architectural Pattern

This project follows the **Model-View-Controller (MVC)** pattern layered on top of **Entity Framework Core** with SQL Server, adopting a **Repository + Unit of Work** abstraction for the data access layer. The architecture enforces strict **Separation of Concerns** across four logical tiers:



Dependency Injection wires every layer together in `Program.cs`. Controllers never instantiate their dependencies; they receive them via constructor injection, making the system testable and loosely coupled.

5.2 Data Access Layer

5.2.1 ApplicationDbContext

The `ApplicationDbContext` extends `IdentityDbContext<ApplicationUser>` to integrate ASP.NET Core Identity tables (AspNetUsers, AspNetRoles, etc.) with the application's domain tables:

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
        : base(options)
    {
    }

    public DbSet<Category> Categories { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<ProductVariant> ProductVariants { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
```

```
    public DbSet<ShoppingCart> ShoppingCarts { get; set; }
    public DbSet<Payment> Payments { get; set; }
    public DbSet<ProductReview> ProductReviews { get; set; }
    public DbSet<PromoCode> PromoCodes { get; set; }
    public DbSet<Wishlist> Wishlists { get; set; }
}
```

Entity relationship configuration is done fluently in `OnModelCreating`. Key decisions:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // Category → Products (1:N) — restrict delete to prevent orphaned
products
    modelBuilder.Entity<Category>()
        .HasMany(c => c.Products)
        .WithOne(p => p.Category)
        .HasForeignKey(p => p.CategoryId)
        .OnDelete(DeleteBehavior.Restrict);

    // Product → Variants (1:N) — cascade delete (variants are worthless
without the product)
    modelBuilder.Entity<Product>()
        .HasMany(p => p.ProductVariants)
        .WithOne(pv => pv.Product)
        .HasForeignKey(pv => pv.ProductId)
        .OnDelete(DeleteBehavior.Cascade);

    // Order → Payment (1:1)
    modelBuilder.Entity<Order>()
        .HasOne(o => o.Payment)
        .WithOne(p => p.Order)
        .HasForeignKey<Payment>(p => p.OrderId)
        .OnDelete(DeleteBehavior.Cascade);

    // Decimal precision for all monetary columns
    modelBuilder.Entity<Product>()
        .Property(p => p.Price).HasPrecision(18, 2);

    // PromoCode unique index
    modelBuilder.Entity<PromoCode>(entity =>
    {
        entity.HasIndex(p => p.Code).IsUnique();
    });

    // Wishlist unique constraint per user+product
    modelBuilder.Entity<Wishlist>(entity =>
    {
        entity.HasIndex(w => new { w.UserId, w.ProductId }).IsUnique();
```

```
        });
    }
```

The `DeleteBehavior.Restrict` on Category→Products is intentional: it prevents accidental deletion of a category that has active products. The admin must reassign or delete the products first, enforcing data integrity at the database level.

5.2.2 Generic Repository Pattern

The `IRepository<T>` interface abstracts all CRUD operations, providing a consistent data access contract:

```
public interface IRepository<T> where T : class
{
    IQueryable<T> GetQueryable(bool asNoTracking);

    Task<IEnumerable<T>> GetAllAsync();
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter);
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter, params
Expression<Func<T, object>>[] includes);
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter, bool
asNoTracking, params Expression<Func<T, object>>[] includes);

    Task<T?> GetByIdAsync(int id);
    Task<T?> GetByIdAsync(int id, bool asNoTracking, params
Expression<Func<T, object>>[] includes);

    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter);
    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter, bool
asNoTracking, params Expression<Func<T, object>>[] includes);

    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
    void DeleteRange(IEnumerable<T> entities);

    Task<bool> AnyAsync(Expression<Func<T, bool>> filter);
    Task<int> CountAsync(Expression<Func<T, bool>>? filter = null);
}
```

The concrete `Repository<T>` class implements each method against EF Core:

```
public class Repository<T> : IRepository<T> where T : class
{
    private readonly ApplicationDbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repository(ApplicationDbContext context)
    {
        _context = context;
```

```
        _dbSet = context.Set<T>();
    }

    public IQueryable<T> GetQueryable(bool asNoTracking)
    {
        return asNoTracking ? _dbSet.AsNoTracking() : _dbSet;
    }

    private IQueryable<T> ApplyIncludes(IQueryable<T> query, params
Expression<Func<T, object>>[] includes)
    {
        if (includes is { Length: > 0 })
        {
            foreach (var include in includes)
            {
                query = query.Include(include);
            }
        }
        return query;
    }

    public async Task<T?> GetByIdAsync(int id, bool asNoTracking, params
Expression<Func<T, object>>[] includes)
    {
        var query = GetQueryable(asNoTracking);
        query = ApplyIncludes(query, includes);
        return await query.FirstOrDefaultAsync(e => EF.Property<int>(e,
"Id") == id);
    }

    // ... remaining implementations follow the same pattern
}
```

AsNoTracking overload design. Every read method has an `asNoTracking` parameter (defaulting to `false` for the simpler overloads). This is deliberate:

- **Read-only GET operations** pass `asNoTracking: true` to avoid the EF Core change tracker overhead, improving performance and reducing memory usage.
- **Write operations** (add, update, delete) pass nothing (tracking on), ensuring EF Core correctly detects changes.
- The `GetQueryable` base method lets callers compose further LINQ operations (`.Where()`, `.OrderBy()`, `.Skip()`, `.Take()`) while still controlling tracking behavior.

5.2.3 Unit of Work

The `IUnitOfWork` interface exposes one repository per entity type and two transaction methods:

```
public interface IUnitOfWork : IDisposable
{
    IRepository<Category> Categories { get; }
    IRepository<Product> Products { get; }
```

```
    IRepository<ProductVariant> ProductVariants { get; }
    IRepository<Order> Orders { get; }
    IRepository<OrderItem> OrderItems { get; }
    IRepository<ShoppingCart> ShoppingCarts { get; }
    IRepository<Payment> Payments { get; }
    IRepository<ApplicationUser> Users { get; }
    IRepository<ProductReview> ProductReviews { get; }
    IRepository<PromoCode> PromoCodes { get; }
    IRepository<Wishlist> Wishlists { get; }

    Task<int> SaveAsync();
    Task<IDbContextTransaction> BeginTransactionAsync();
}
```

The concrete `UnitOfWork` class instantiates each repository with the shared `DbContext`, ensuring all operations within a single request share the same change tracker:

```
public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;

    public IRepository<Category> Categories { get; private set; }
    public IRepository<Product> Products { get; private set; }
    // ... all repositories

    public UnitOfWork(ApplicationDbContext context)
    {
        _context = context;
        Categories = new Repository<Category>(_context);
        Products = new Repository<Product>(_context);
        // ... initialize all repositories
    }

    public async Task<int> SaveAsync()
    {
        return await _context.SaveChangesAsync();
    }

    public async Task<IDbContextTransaction> BeginTransactionAsync()
    {
        return await _context.Database.BeginTransactionAsync();
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

Why Unit of Work? Without it, each repository would need its own `DbContext` instance, making cross-entity transactions impossible. With UoW, controllers call `SaveAsync()` once at the end of a request, and if any operation fails, the entire batch rolls back. The `BeginTransactionAsync()` method enables explicit database transactions for critical paths like checkout.

5.2.4 DI Registration in Program.cs

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConn
ection")));

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
```

`AddScoped` ensures one `UnitOfWork` (and therefore one `DbContext`) per HTTP request — the standard lifetime for EF Core in web applications.

5.3 Entities and Database Schema

5.3.1 ApplicationUser (Extended Identity)

```
public class ApplicationUser : IdentityUser
{
    public string FullName { get; set; } = string.Empty;
    public string? Address { get; set; }
    public string? City { get; set; }
    public string? Country { get; set; }
    public DateTime CreatedDate { get; set; } = DateTime.Now;

    // Navigation Properties
    public virtual ICollection<Order> Orders { get; set; } = new
List<Order>();
    public virtual ICollection<ShoppingCart> ShoppingCarts { get; set; } =
new List<ShoppingCart>();
}
```

Extends the `IdentityUser` base class (which provides Id, UserName, Email, PasswordHash, etc.) with profile information needed for shipping and display. The `Orders` and `ShoppingCarts` navigation properties enable LINQ queries like `user.Orders.Where(o => o.Status == OrderStatus.Pending)`.

5.3.2 Category

```
[Key]
public int Id { get; set; }

[Required(ErrorMessage = "اسم الفئة مطلوب")]
[StringLength(100)]
```

```
public string Name { get; set; } = string.Empty;

[StringLength(500)]
public string? Description { get; set; }

public string? ImageUrl { get; set; }
public DateTime CreatedDate { get; set; } = DateTime.Now;
public bool IsActive { get; set; } = true;

public virtual ICollection<Product> Products { get; set; } = new
List<Product>();
```

5.3.3 Product (with Concurrency Token)

```
public class Product
{
    public int Id { get; set; }

    [Required, StringLength(200)]
    public string Name { get; set; } = string.Empty;

    [StringLength(2000)]
    public string? Description { get; set; }

    [Required, Column(TypeName = "decimal(18,2)"), Range(0.01,
double.MaxValue)]
    public decimal Price { get; set; }

    [Required, Range(0, int.MaxValue)]
    public int Stock { get; set; }

    public string? ImageUrl { get; set; }

    [Required]
    public int CategoryId { get; set; }

    public DateTime CreatedDate { get; set; } = DateTime.Now;
    public bool IsActive { get; set; } = true;
    public bool IsFeatured { get; set; } = false;

    [Timestamp]
    public byte[] RowVersion { get; set; } = [];

    // Navigation Properties
    public virtual Category Category { get; set; } = null!;
    public virtual ICollection<OrderItem> OrderItems { get; set; } = new
List<OrderItem>();
    public virtual ICollection<ShoppingCart> ShoppingCarts { get; set; } =
new List<ShoppingCart>();
    public virtual ICollection<ProductVariant> ProductVariants { get; set;
```

```
} = new List<ProductVariant>();
}
```

`[Timestamp]` **RowVersion.** This is the concurrency token that EF Core maps to SQL Server's `rowversion` type. Every time the row is updated, SQL Server automatically increments the rowversion value. When two users attempt to update the same product simultaneously, EF Core detects the mismatch between the original and current rowversion values and throws a `DbUpdateConcurrencyException`. This is critical for stock management during checkout (see §5.7).

5.3.4 ProductVariant

```
public class ProductVariant
{
    public int Id { get; set; }
    [Required]
    public int ProductId { get; set; }

    [StringLength(50)]
    public string? Size { get; set; } // S, M, L, XL

    [StringLength(50)]
    public string? Color { get; set; }

    [Column(TypeName = "decimal(18,2)")]
    public decimal AdditionalPrice { get; set; } = 0;

    public int Stock { get; set; } = 0;

    [Timestamp]
    public byte[] RowVersion { get; set; } = [];

    public bool IsActive { get; set; } = true;

    public virtual Product Product { get; set; } = null!;
}
```

Variants allow a product to have different size/color combinations, each with its own stock and price delta. The concurrency token on both `Product` and `ProductVariant` prevents race conditions when two shoppers buy the last unit of a specific size.

5.3.5 ShoppingCart

```
public class ShoppingCart
{
    public int Id { get; set; }
    [Required]
    public string UserId { get; set; } = string.Empty;
    [Required]
```

```
    public int ProductId { get; set; }
    [Required]
    public int Quantity { get; set; } = 1;
    public int? ProductVariantId { get; set; }
    public DateTime AddedDate { get; set; } = DateTime.Now;

    public virtual ApplicationUser User { get; set; } = null!;
    public virtual Product Product { get; set; } = null!;
    public virtual ProductVariant? ProductVariant { get; set; }
}
```

Composite key logic is handled in application code (not the database): a user can have at most one cart entry per `(ProductId, VariantId)` combination. This is enforced by the `AddToCart` method which first checks for an existing entry and increments quantity if found, rather than inserting a duplicate.

5.3.6 Order and OrderItem

```
public class Order
{
    public int Id { get; set; }
    [Required]
    public string UserId { get; set; } = string.Empty;
    public DateTime OrderDate { get; set; } = DateTime.Now;

    [Required, Column(TypeName = "decimal(18,2)")]
    public decimal TotalAmount { get; set; }

    [Required]
    public OrderStatus Status { get; set; }  // enum: Pending, Paid,
Processing, Shipped, Delivered, Cancelled, Refunded

    [Required]
    public PaymentMethod PaymentMethod { get; set; } // enum:
CashOnDelivery, CreditCard

    [Required, StringLength(500)]
    public string ShippingAddress { get; set; } = string.Empty;
    [Required, StringLength(100)]
    public string City { get; set; } = string.Empty;
    [Required, StringLength(100)]
    public string Country { get; set; } = string.Empty;
    public string? State { get; set; }
    public string? ZipCode { get; set; }
    [Required, Phone]
    public string PhoneNumber { get; set; } = string.Empty;
    public string Notes { get; set; } = string.Empty;
    public DateTime? DeliveredDate { get; set; }

    // PromoCode
    public int? PromoCodeId { get; set; }
    [Column(TypeName = "decimal(18,2)")]
```

```
    public decimal DiscountAmount { get; set; } = 0;

    public virtual ApplicationUser? User { get; set; }
    public virtual PromoCode? PromoCode { get; set; }
    public virtual ICollection<OrderItem> OrderItems { get; set; } = new
List<OrderItem>();
    public virtual Payment? Payment { get; set; }
}

public class OrderItem
{
    public int Id { get; set; }
    [Required]
    public int OrderId { get; set; }
    [Required]
    public int ProductId { get; set; }
    [Required]
    public int Quantity { get; set; }
    [Required, Column(TypeName = "decimal(18,2)")]
    public decimal UnitPrice { get; set; }
    [Required, Column(TypeName = "decimal(18,2)")]
    public decimal TotalPrice { get; set; }

    public virtual Order Order { get; set; } = null!;
    public virtual Product Product { get; set; } = null!;
}
```

The `UnitPrice` is stored at order time rather than computed from the current product price — this is an intentional snapshot pattern. If the admin changes a product's price later, past orders still reflect the price the customer actually paid. `TotalPrice = UnitPrice * Quantity` is precomputed for efficient analytics queries without runtime multiplication.

5.3.7 OrderStatus Enum

```
public enum OrderStatus
{
    Pending = 1,
    Paid = 2,
    Processing = 3,
    Shipped = 4,
    Delivered = 5,
    Cancelled = 6,
    Refunded = 7
}
```

The status progression is: `Pending → Paid → Processing → Shipped → Delivered`. Cancellation is possible from Pending or Paid states. The `Refunded` status terminates the lifecycle after delivery.

5.3.8 PaymentMethod Enum

```
public enum PaymentMethod
{
    CashOnDelivery = 1,
    CreditCard = 2
}
```

5.3.9 PromoCode

```
public class PromoCode
{
    public int Id { get; set; }
    [Required, StringLength(50)]
    public string Code { get; set; } = string.Empty;

    public DiscountType DiscountType { get; set; }  // Percentage or
FixedAmount
    [Required]
    public decimal DiscountValue { get; set; }

    public decimal? MinimumPurchase { get; set; }
    public decimal? MaximumDiscount { get; set; }  // cap for percentage
discounts
    public DateTime? StartDate { get; set; }
    public DateTime? EndDate { get; set; }

    public int? UsageLimit { get; set; }        // global cap
    public int UsageCount { get; set; } = 0;
    public int? UsageLimitPerUser { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; } = [];

    public bool IsActive { get; set; } = true;
    public DateTime CreatedDate { get; set; } = DateTime.Now;

    public virtual ICollection<Order> Orders { get; set; } = new
List<Order>();
}
```

The `RowVersion` on PromoCode prevents the race condition where two users apply the same limited-use promo code simultaneously, potentially allowing usage beyond the `UsageLimit`. The checkout transaction (§5.7) handles this with a retry loop.

5.3.10 Database Diagram (Relational Summary)

```
Users (AspNetUsers)
  ├── Orders (UserId FK, Restrict)
```

```
    │     ├── OrderItems (OrderId FK, Cascade)
    │     │     └── Products (ProductId FK, Restrict)
    │     │           ├── Categories (CategoryId FK, Restrict)
    │     │           └── ProductVariants (ProductId FK, Cascade)
    │     ├── Payments (OrderId FK, Cascade, 1:1)
    │     └── PromoCodes (PromoCodeId FK, SetNull)
    └── ShoppingCarts (UserId FK, Cascade)
          └── Products (ProductId FK)
```

5.4 Business Logic — Product & Category Management (Admin)

5.4.1 Product CRUD with Image Upload

The `AdminController` handles all product management. The `CreateProduct` action demonstrates the pattern:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CreateProduct(Product product, IFormFile?
ImageFile)
{
    ModelState.Remove("Category");
    ModelState.Remove("OrderItems");
    ModelState.Remove("ShoppingCarts");
    ModelState.Remove("ProductVariants");

    if (!ModelState.IsValid)
    {
        var categories = await _unitOfWork.Categories.GetAsync(c =>
c.IsActive);
        ViewBag.Categories = categories.ToList();
        return View(product);
    }

    try
    {
        if (ImageFile != null && ImageFile.Length > 0)
        {
            product.ImageUrl = await
_imageService.UploadImageAsync(ImageFile, "products");
        }

        product.CreatedDate = DateTime.Now;
        product.IsActive = true;

        await _unitOfWork.Products.AddAsync(product);
        await _unitOfWork.SaveAsync();

        TempData["SuccessMessage"] = "Product created successfully!";
        return RedirectToAction(nameof(Products));
    }
```

```
        catch (Exception ex)
        {
            _logger.LogError(ex, "AdminController error");
            ModelState.AddModelError("", $"Error: {ex.Message}");
            // Re-populate categories for the failed form
            var categories = await _unitOfWork.Categories.GetAsync(c =>
c.IsActive);
            ViewBag.Categories = categories.ToList();
            return View(product);
        }
    }
```

Key details:

- `ModelState.Remove(...)` strips navigation properties from validation — the Category navigation property is populated by EF after save, not from the form.
- Image upload delegates to `IImageService`; if no file is provided, `ImageUrl` remains null.
- Success/failure follows the Post-Redirect-Get (PRG) pattern using `TempData`.

5.4.2 Image Service

```
public class ImageService : IImageService
{
    private readonly IWebHostEnvironment _environment;
    private readonly ILogger<ImageService> _logger;

    public ImageService(IWebHostEnvironment environment,
ILogger<ImageService> logger)
    {
        _environment = environment;
        _logger = logger;
    }

    public async Task<string> UploadImageAsync(IFormFile file, string
folder)
    {
        if (file == null || file.Length == 0)
            return string.Empty;

        try
        {
            var uploadsFolder = Path.Combine(_environment.WebRootPath,
"images", folder);

            if (!Directory.Exists(uploadsFolder))
                Directory.CreateDirectory(uploadsFolder);

            var fileName = Guid.NewGuid().ToString() +
Path.GetExtension(file.FileName);
            var filePath = Path.Combine(uploadsFolder, fileName);
```

```
            using (var stream = new FileStream(filePath, FileMode.Create))
            {
                await file.CopyToAsync(stream);
            }

            return $"/images/{folder}/{fileName}";
        }
        catch (Exception ex)
        {
            _logger.LogError($"Error uploading image: {ex.Message}");
            throw;
        }
    }

    public async Task<bool> DeleteImageAsync(string imagePath)
    {
        if (string.IsNullOrEmpty(imagePath))
            return true;

        try
        {
            var fullPath = Path.Combine(_environment.WebRootPath,
imagePath.TrimStart('/'));
            if (File.Exists(fullPath))
                File.Delete(fullPath);
            return true;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error deleting image {ImagePath}",
imagePath);
            return false;
        }
    }
}
```

Design decisions:

- `Guid.NewGuid()` generates a unique filename — prevents collisions and path-traversal attacks.
- The original extension is preserved for browser compatibility.
- Files are stored under `wwwroot/images/{folder}/` so they are served directly by the static file middleware without a controller action.
- Deletion silently returns `true` even if the file doesn't exist (idempotent).

5.5 Performance Optimization — In-Memory Caching

5.5.1 Registration

```
builder.Services.AddMemoryCache();
builder.Services.AddResponseCaching();
```

`IMemoryCache` is used for server-side caching of expensive, infrequently-changed data. `ResponseCaching` is separately configured for static pages like Privacy and Terms.

5.5.2 Cached Data Sources

Three data sets are cached to reduce database load:

HomePage featured products (HomeController):

```
var featuredProducts = await
_memoryCache.GetOrCreateAsync("FeaturedProducts", async entry =>
{
    entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
    entry.SlidingExpiration = TimeSpan.FromMinutes(2);
    return await _unitOfWork.Products.GetQueryable(asNoTracking: true)
        .Where(p => p.IsFeatured && p.IsActive)
        .OrderBy(p => p.Id)
        .Take(8)
        .ToListAsync();
});
```

Navigation categories (_Layout.cshtml via IMemoryCache injection):

```
@inject IMemoryCache MemoryCache
@inject IUnitOfWork UnitOfWork
@{
    var navCategories = await MemoryCache.GetOrCreateAsync("NavCategories",
async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
        entry.SlidingExpiration = TimeSpan.FromMinutes(2);
        return (await UnitOfWork.Categories.GetAsync(c =>
c.IsActive)).ToList();
    });
}
```

Product listing categories (ProductsController):

```
private async Task<List<Category>> GetCachedCategoriesAsync()
{
    return await _memoryCache.GetOrCreateAsync("ProductCategories", async
entry =>
    {
```

```
            entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
            entry.SlidingExpiration = TimeSpan.FromMinutes(2);
            return (await _unitOfWork.Categories.GetAsync(c => c.IsActive,
asNoTracking: true)).ToList();
        }) ?? [];
    }
```

Cache eviction strategy:

- **Absolute expiration (10 minutes)** — guarantees stale data is eventually refreshed even without activity.
- **Sliding expiration (2 minutes)** — extends the cache lifetime as long as the data is being accessed frequently, preventing cache churn under load.
- Both conditions must expire: the cache entry lives until either 10 minutes from creation OR 2 minutes of inactivity, whichever expires last.

5.6 Cart Logic with Stock Validation

5.6.1 AddToCart — Server-Side Stock Check

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> AddToCart(int productId, int quantity = 1,
int? variantId = null)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (string.IsNullOrEmpty(userId))
        return RedirectToAction("Login", "Account");

    // 1. Verify product exists and is active
    var product = await _unitOfWork.Products.GetByIdAsync(productId);
    if (product == null || !product.IsActive) { /* redirect with error */ }

    // 2. If variant selected, verify it and check its stock
    ProductVariant? selectedVariant = null;
    if (variantId.HasValue)
    {
        selectedVariant = await
_unitOfWork.ProductVariants.GetFirstOrDefaultAsync(
            v => v.Id == variantId && v.ProductId == productId &&
v.IsActive);
        if (selectedVariant == null) { /* error */ }
        if (selectedVariant.Stock < quantity) { /* error */ }
    }

    // 3. Check base stock
    var availableStock = selectedVariant?.Stock ?? product.Stock;
    if (availableStock < quantity) { /* error */ }

    // 4. Check for existing cart entry
    var existingCartItem = await
```

```
_unitOfWork.ShoppingCarts.GetFirstOrDefaultAsync(
        c => c.UserId == userId && c.ProductId == productId &&
c.ProductVariantId == variantId);

    if (existingCartItem != null)
    {
        // Increment, checking cumulative stock
        existingCartItem.Quantity += quantity;
        if (existingCartItem.Quantity > availableStock) { /* error */ }
        _unitOfWork.ShoppingCarts.Update(existingCartItem);
    }
    else
    {
        // New entry
        await _unitOfWork.ShoppingCarts.AddAsync(new ShoppingCart { ... });
    }

    await _unitOfWork.SaveAsync();
    TempData["SuccessMessage"] = $"{product.Name} added to cart
successfully!";
    return RedirectToAction(nameof(Index));
}
```

Validation flow: product active → variant valid → individual stock sufficient → cumulative cart quantity within stock. Each check returns a specific error message to the user, preventing overselling at the add-to-cart stage.

5.6.2 Quantity Update with Stock Boundary

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> UpdateQuantity(int cartId, int quantity)
{
    var cartItem = await _unitOfWork.ShoppingCarts.GetByIdAsync(cartId);
    if (cartItem == null || cartItem.UserId != userId) { /* error */ }

    var product = await
_unitOfWork.Products.GetByIdAsync(cartItem.ProductId);
    if (quantity > product.Stock) { /* error: "Only N units available" */ }

    if (quantity <= 0)
    {
        _unitOfWork.ShoppingCarts.Delete(cartItem); // Remove if quantity
drops to zero
    }
    else
    {
        cartItem.Quantity = quantity;
        _unitOfWork.ShoppingCarts.Update(cartItem);
    }
```

```
        await _unitOfWork.SaveAsync();
    }
```

Quantity ≤ 0 triggers deletion rather than storing a zero-quantity cart item, keeping the data clean.

5.7 Checkout, Concurrency, and Payments

5.7.1 Checkout Index — Pre-fill User Profile

```
public async Task<IActionResult> Index(string? promoCode = null)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    var user = await _userManager.FindByIdAsync(userId);

    var model = new CheckoutViewModel
    {
        FullName = user?.FullName ?? "",
        Email = user?.Email ?? "",
        PhoneNumber = user?.PhoneNumber ?? "",
        Address = user?.Address ?? "",
        City = user?.City ?? "",
        Country = user?.Country ?? "",
        PromoCode = promoCode
    };

    // Cart summary for display
    var cartItems = await
_unitOfWork.ShoppingCarts.GetQueryable(asNoTracking: true)
        .Where(c => c.UserId == userId)
        .Include(c => c.Product)
        .ToListAsync();

    var subtotal = cartItems.Sum(item => item.Product.Price *
item.Quantity);
    ViewBag.Subtotal = subtotal;
    ViewBag.Tax = subtotal * 0.14m;
    ViewBag.Total = subtotal * 1.14m;

    return View(model);
}
```

The form is pre-populated from the user's profile, reducing friction for returning customers.

5.7.2 PlaceOrder — The Transaction Kernel

This is the most critical method in the application. It orchestrates: stock deduction, promo code application, order creation, payment record creation, and cart cleanup — all within a single database transaction with concurrency conflict retry.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> PlaceOrder(CheckoutViewModel model)
{
    const int maxRetries = 3;

    for (int attempt = 1; attempt <= maxRetries; attempt++)
    {
        await using var transaction = await
_unitOfWork.BeginTransactionAsync();

        try
        {
            // --- Step 1: Load cart ---
            var cartItems = await _unitOfWork.ShoppingCarts.GetAsync(
                c => c.UserId == userId, c => c.Product);
            if (!cartItems.Any()) { /* redirect: cart empty */ }

            var cartItemList = cartItems.ToList();

            // --- Step 2: Aggregate ALL stock failures before processing -
--
            var outOfStockItems = cartItemList
                .Where(ci => ci.Product == null || !ci.Product.IsActive ||
ci.Product.Stock < ci.Quantity)
                .Select(ci => ci.Product?.Name ?? "Unknown")
                .ToList();

            if (outOfStockItems.Any())
            {
                await transaction.RollbackAsync();
                foreach (var item in outOfStockItems)
                    ModelState.AddModelError(string.Empty, $"\"{item}\"
stock insufficient.");
                return View("Index", model);
            }

            // --- Step 3: Deduct stock and build OrderItems ---
            decimal subtotal = 0;
            var orderItems = new List<OrderItem>();

            foreach (var cartItem in cartItemList)
            {
                var product = cartItem.Product;
                if (product == null || !product.IsActive) continue;

                var itemTotal = product.Price * cartItem.Quantity;
                subtotal += itemTotal;

                orderItems.Add(new OrderItem
                {
                    ProductId = product.Id,
```

```
                    Quantity = cartItem.Quantity,
                    UnitPrice = product.Price,
                    TotalPrice = itemTotal
                });

                product.Stock -= cartItem.Quantity;  // Deduct stock
                _unitOfWork.Products.Update(product);  // Triggers
concurrency token check on SaveAsync
            }

            // --- Step 4: Calculate totals ---
            decimal tax = subtotal * 0.14m;
            decimal totalAmount = subtotal + tax;

            // --- Step 5: Apply promo code (if provided) ---
            int? promoCodeId = null;
            decimal discountAmount = 0;

            if (!string.IsNullOrWhiteSpace(model.PromoCode))
            {
                var promoCode = await
_unitOfWork.PromoCodes.GetFirstOrDefaultAsync(
                    p => p.Code.ToUpper() == model.PromoCode.ToUpper() &&
p.IsActive);

                if (promoCode != null)
                {
                    bool isValid = ValidatePromoCode(promoCode,
totalAmount);

                    if (isValid)
                    {
                        discountAmount = CalculateDiscount(promoCode,
totalAmount);
                        totalAmount -= discountAmount;
                        promoCodeId = promoCode.Id;

                        promoCode.UsageCount++;  // Increment usage
                        _unitOfWork.PromoCodes.Update(promoCode);  //
Concurrency check
                    }
                }
            }

            // --- Step 6: Create Order ---
            var order = new Order
            {
                UserId = userId,
                OrderDate = DateTime.Now,
                TotalAmount = totalAmount,
                Status = OrderStatus.Pending,
                PaymentMethod = model.PaymentMethod,
                ShippingAddress = model.Address,
                City = model.City, State = model.State,
```

```
                ZipCode = model.ZipCode, Country = model.Country,
                PhoneNumber = model.PhoneNumber,
                Notes = model.Notes ?? string.Empty,
                PromoCodeId = promoCodeId,
                DiscountAmount = discountAmount,
                OrderItems = orderItems
            };

            await _unitOfWork.Orders.AddAsync(order);

            // --- Step 7: Create Payment ---
            var payment = new Payment
            {
                Amount = totalAmount,
                PaymentDate = DateTime.Now,
                PaymentMethod = model.PaymentMethod,
                Status = PaymentStatus.Pending,
                TransactionId = $"PENDING-{DateTime.Now.Ticks}"
            };
            order.Payment = payment;

            await _unitOfWork.SaveAsync();  // Single SaveChanges — ALL or
NOTHING

            // --- Step 8: Handle payment method ---
            if (model.PaymentMethod == PaymentMethod.CashOnDelivery)
            {
                // COD: complete immediately
                _unitOfWork.ShoppingCarts.DeleteRange(cartItems);
                await _unitOfWork.SaveAsync();
                await transaction.CommitAsync();

                // Send confirmation email (fire-and-forget)
                await SendOrderConfirmationEmail(userId, order.Id,
totalAmount);

                return RedirectToAction("OrderConfirmation", new { orderId
= order.Id });
            }

            // Credit Card: redirect to Stripe
            var checkoutUrl = await
_paymentService.CreateCheckoutSessionAsync(
                order.Id, totalAmount,
                cartItemList.Select(ci => ci.Product!.Name).ToList());

            _unitOfWork.ShoppingCarts.DeleteRange(cartItems);
            await _unitOfWork.SaveAsync();
            await transaction.CommitAsync();

            return Redirect(checkoutUrl);
        }
        catch (DbUpdateConcurrencyException) when (attempt < maxRetries)
        {
```

```
            // --- Concurrency conflict: rollback and retry ---
            await transaction.RollbackAsync();
            _logger.LogWarning("Concurrency conflict (attempt
{Attempt}/{MaxRetries})", attempt, maxRetries);
            await Task.Delay(100 * attempt);  // Exponential back-off:
100ms, 200ms, 300ms
            continue;
        }
        catch (DbUpdateConcurrencyException) when (attempt >= maxRetries)
        {
            await transaction.RollbackAsync();
            _logger.LogError("Concurrency conflict — max retries reached");
            TempData["ErrorMessage"] = "Some items were just purchased.
Please review your cart.";
            return RedirectToAction("Index");
        }
        catch (DbUpdateException dbEx)
        {
            await transaction.RollbackAsync();
            _logger.LogError(dbEx, "Database error placing order");
            ModelState.AddModelError(string.Empty, $"DB ERROR:
{dbEx.InnerException?.Message ?? dbEx.Message}");
            return View("Index", model);
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;  // Let the global error handler process this
        }
    }

    return RedirectToAction("Index");
}
```

5.7.3 Concurrency Handling in Detail

The `Product.Stock` field and `PromoCode.UsageCount` field both have `[Timestamp] byte[] RowVersion` attributes. When the checkout calls `_unitOfWork.SaveAsync()`, EF Core generates an SQL UPDATE like:

```
UPDATE Products SET Stock = @NewStock
WHERE Id = @Id AND RowVersion = @OriginalRowVersion;

UPDATE PromoCodes SET UsageCount = @NewCount
WHERE Id = @Id AND RowVersion = @OriginalRowVersion;
```

If another request has already modified either row between the SELECT and UPDATE, the `WHERE RowVersion = @OriginalRowVersion` matches zero rows, and EF Core throws `DbUpdateConcurrencyException`.

The retry loop handles this:

1. **Rollback** the entire transaction (all changes discarded).
2. **Wait** (100ms × attempt number) — exponential back-off reduces retry storms.
3. **Re-read** fresh data (new SELECT inside the next iteration).
4. **Retry** the entire order placement.

After 3 consecutive failures, the user receives a clear message asking them to review their cart — the items they wanted were purchased by someone else in the seconds between adding to cart and checking out.

Why aggregate stock failures (§5.7.2 Step 2)? All stock checks are performed *before* any writes. If multiple products are out of stock, the user sees *all* the failures at once, rather than fixing one at a time.

5.7.4 Promo Code Validation (Server-Side)

The validation endpoint called by the checkout page before order placement:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> ValidatePromoCode([FromBody]
ValidatePromoCodeRequest request)
{
    var promoCode = await _unitOfWork.PromoCodes.GetFirstOrDefaultAsync(
        p => p.Code.ToUpper() == request.Code.ToUpper());

    if (promoCode == null)
        return Json(new { success = false, message = "Invalid promo code."
});

    if (!promoCode.IsActive)
        return Json(new { success = false, message = "This promo code is no
longer active." });

    if (promoCode.StartDate.HasValue && DateTime.Now <
promoCode.StartDate.Value)
        return Json(new { success = false, message = "This promo code is
not yet valid." });

    if (promoCode.EndDate.HasValue && DateTime.Now >
promoCode.EndDate.Value)
        return Json(new { success = false, message = "This promo code has
expired." });

    if (promoCode.UsageLimit.HasValue && promoCode.UsageCount >=
promoCode.UsageLimit.Value)
        return Json(new { success = false, message = "This promo code has
reached its usage limit." });

    if (promoCode.MinimumPurchase.HasValue && request.OrderTotal <
promoCode.MinimumPurchase.Value)
        return Json(new { success = false, message = $"Minimum purchase of
{promoCode.MinimumPurchase.Value:C} required." });
```

```
    // Calculate discount
    decimal discountAmount = 0;
    if (promoCode.DiscountType == DiscountType.Percentage)
    {
        discountAmount = request.OrderTotal * (promoCode.DiscountValue /
100);
        if (promoCode.MaximumDiscount.HasValue && discountAmount >
promoCode.MaximumDiscount.Value)
            discountAmount = promoCode.MaximumDiscount.Value;
    }
    else
    {
        discountAmount = promoCode.DiscountValue;
    }

    if (discountAmount > request.OrderTotal)
        discountAmount = request.OrderTotal;

    var newTotal = request.OrderTotal - discountAmount;

    return Json(new
    {
        success = true,
        discountAmount,
        newTotal,
        message = $"You saved {discountAmount:C}"
    });
}
```

Five validation checks: existence, active flag, start date, end date, usage limit, and minimum purchase. The `DiscountType` enum determines whether the value is a percentage (capped by `MaximumDiscount`) or a fixed amount. The discount can never exceed the order total.

5.7.5 Stripe Payment Integration

```
public class StripePaymentService : IPaymentService
{
    private readonly string _secretKey;
    private readonly string _domain;

    public StripePaymentService(IConfiguration configuration,
ILogger<StripePaymentService> logger)
    {
        _secretKey = configuration["Stripe:SecretKey"] ?? throw new
ArgumentNullException("Stripe SecretKey");
        _domain = configuration["Stripe:Domain"] ?? throw new
ArgumentNullException("Stripe Domain");
        StripeConfiguration.ApiKey = _secretKey;
    }
```

```
    public async Task<string> CreateCheckoutSessionAsync(int orderId,
decimal amount, List<string> productNames)
    {
        var lineItems = new List<SessionLineItemOptions>
        {
            new SessionLineItemOptions
            {
                PriceData = new SessionLineItemPriceDataOptions
                {
                    Currency = "usd",
                    ProductData = new
SessionLineItemPriceDataProductDataOptions
                    {
                        Name = $"Order #{orderId}",
                        Description = string.Join(", ",
productNames.Take(3))
                    },
                    UnitAmount = (long)(amount * 100),  // Stripe uses
cents
                },
                Quantity = 1,
            }
        };

        var options = new SessionCreateOptions
        {
            PaymentMethodTypes = new List<string> { "card" },
            LineItems = lineItems,
            Mode = "payment",
            SuccessUrl = $"{_domain}/Checkout/PaymentSuccess?orderId=
{orderId}",
            CancelUrl = $"{_domain}/Checkout/PaymentCancelled?orderId=
{orderId}",
            Metadata = new Dictionary<string, string>
            {
                { "order_id", orderId.ToString() }
            }
        };

        var service = new SessionService();
        var session = await service.CreateAsync(options);
        return session.Url;
    }
}
```

Key details:

- Stripe operates in cents (smallest currency unit). The `amount * 100` converts USD dollars to cents.
- `SuccessUrl` and `CancelUrl` define what happens after the Stripe-hosted checkout page.
- The `Metadata` dictionary carries the order ID through the redirect, enabling the `PaymentSuccess` action to identify which order was paid.

- The `IPaymentService` interface abstracts Stripe-specific code, allowing future payment providers (e.g., PayPal) without changing the controller.

5.7.6 Payment Callback Handling

PaymentSuccess: Updates payment status to `Completed`, order status to `Paid`, sends confirmation email.

```
public async Task<IActionResult> PaymentSuccess(int orderId)
{
    var order = await _unitOfWork.Orders.GetByIdAsync(orderId);
    var payment = await _unitOfWork.Payments.GetFirstOrDefaultAsync(p =>
p.OrderId == orderId);

    payment.Status = PaymentStatus.Completed;
    payment.PaymentDate = DateTime.Now;
    _unitOfWork.Payments.Update(payment);

    order.Status = OrderStatus.Paid;
    _unitOfWork.Orders.Update(order);
    await _unitOfWork.SaveAsync();

    // Send confirmation email
    await SendOrderConfirmationEmail(userId, order.Id, order.TotalAmount);

    return RedirectToAction("OrderConfirmation", new { orderId });
}
```

PaymentCancelled: Sets payment to `Failed`, order to `Cancelled`, allowing the user to retry.

```
public async Task<IActionResult> PaymentCancelled(int orderId)
{
    var order = await _unitOfWork.Orders.GetByIdAsync(orderId);
    var payment = await _unitOfWork.Payments.GetFirstOrDefaultAsync(p =>
p.OrderId == orderId);

    payment.Status = PaymentStatus.Failed;
    _unitOfWork.Payments.Update(payment);

    order.Status = OrderStatus.Cancelled;
    _unitOfWork.Orders.Update(order);
    await _unitOfWork.SaveAsync();

    TempData["ErrorMessage"] = "Payment was cancelled. Please try again.";
    return RedirectToAction("Index");
}
```

5.8 Security and Error Handling

5.8.1 ASP.NET Core Identity

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    // Password policy
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;

    // Email uniqueness
    options.User.RequireUniqueEmail = true;

    // Sign-in
    options.SignIn.RequireConfirmedEmail = false;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();  // Email confirmation, password reset tokens

// Cookie configuration
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.LogoutPath = "/Account/Logout";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.SlidingExpiration = true;
});
```

Password policy balances security with usability: requires digit + lowercase + uppercase (at least 6 characters). The non-alphanumeric requirement is relaxed since the other rules provide sufficient entropy for an e-commerce application.

The 30-day sliding cookie means users stay logged in across return visits as long as they are active within the 30-day window.

5.8.2 Role-Based Authorization

Admin-only actions are protected with `[Authorize(Roles = "Admin")]`:

```
[Authorize(Roles = "Admin")]
public class AdminController : Controller { ... }
```

The AI Assistant API requires any authenticated user (`[Authorize]` without a role), as discussed in §4.4.2.

5.8.3 Anti-Forgery Tokens

All POST endpoints use `[ValidateAntiForgeryToken]` paired with `@Html.AntiForgeryToken()` in the form. AJAX POSTs (wishlist, promo code validation, add-to-cart) include the token in the request header:

```
// Server: standard ValidateAntiForgeryToken
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> AddToCart(...)

// Client: token extracted from hidden field
function getToken() {
    return
document.querySelector('input[name="__RequestVerificationToken"]')?.value;
}

fetch(url, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'RequestVerificationToken': token
    },
    body: JSON.stringify({ ... })
});
```

5.8.4 Global Error Handling

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

In production, unhandled exceptions are caught by the `UseExceptionHandler` middleware, which redirects to a user-friendly error page without exposing stack traces. HSTS enforces HTTPS in production.

Per-controller error handling is implemented in critical actions:

- **CheckoutController.PlaceOrder** — catches `DbUpdateConcurrencyException`, `DbUpdateException`, and general exceptions with rollback and specific user messages.
- **ProductsController.AddReview** — catches exceptions and returns a generic "Failed to submit review" message.
- **AIAssistantController.Ask** — returns structured JSON error responses.

5.8.5 Input Validation

Server-side validation is enforced through Data Annotations on ViewModels:

```
public class CheckoutViewModel
{
    [Required(ErrorMessage = "Full name is required")]
    [StringLength(100)]
```

```
    public string FullName { get; set; } = string.Empty;

    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required, Phone]
    public string PhoneNumber { get; set; } = string.Empty;

    [Required(ErrorMessage = "Address is required")]
    [StringLength(500)]
    public string Address { get; set; } = string.Empty;

    // ... more fields
}
```

Client-side validation is enabled via the `_ValidationScriptsPartial` partial view, which includes jQuery Validation and Unobtrusive Validation scripts. This provides instant feedback without a server round-trip, while the server-side `ModelState.IsValid` check ensures security (client validation can be bypassed).

5.9 Database Seeding

The `DbInitializer.SeedAsync` method populates the database on first run:

```
public static async Task SeedAsync(ApplicationDbContext context,
    UserManager<ApplicationUser> userManager,
    RoleManager<IdentityRole> roleManager)
{
    // 1. Create roles: Admin, Seller, Customer
    if (!await roleManager.RoleExistsAsync("Admin"))
        await roleManager.CreateAsync(new IdentityRole("Admin"));
    if (!await roleManager.RoleExistsAsync("Seller"))
        await roleManager.CreateAsync(new IdentityRole("Seller"));
    if (!await roleManager.RoleExistsAsync("Customer"))
        await roleManager.CreateAsync(new IdentityRole("Customer"));

    // 2. Create admin user (idempotent — checks email before creating)
    if (await userManager.FindByEmailAsync("admin@ecommerce.com") == null)
    {
        var adminUser = new ApplicationUser { ... };
        await userManager.CreateAsync(adminUser, "Admin@123");
        await userManager.AddToRoleAsync(adminUser, "Admin");
    }

    // 3. Seed 6 categories with 18 products (3 per category)
    if (!context.Categories.Any())
    {
        var categories = new List<Category> { ... };
        context.Categories.AddRange(categories);
        await context.SaveChangesAsync();
    }
```

```
    if (!context.Products.Any())
    {
        var products = new List<Product> { ... };
        context.Products.AddRange(products);
        await context.SaveChangesAsync();
    }
}
```

Called from `Program.cs` with error logging:

```
using (var scope = app.Services.CreateScope())
{
    var context =
scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    var userManager =
scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
    var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    await DbInitializer.SeedAsync(context, userManager, roleManager);
}
```

The seed logic is **idempotent** — it checks for existing data before inserting, allowing the application to restart without duplicate data errors.

5.10 Email Service (Transactional Emails)

The `EmailService` uses SMTP via Gmail for sending transactional emails:

```
public class EmailService : IEmailService
{
    private readonly EmailSettings _emailSettings;

    public EmailService(IOptions<EmailSettings> emailSettings)
    {
        _emailSettings = emailSettings.Value;
    }

    public async Task SendEmailAsync(string toEmail, string subject, string
body)
    {
        var smtpClient = new SmtpClient(_emailSettings.SMTPServer)
        {
            Port = _emailSettings.SMTPPort,
            Credentials = new NetworkCredential(_emailSettings.SenderEmail,
_emailSettings.SenderPassword),
            EnableSsl = true,
        };
```

```
        var mailMessage = new MailMessage
        {
            From = new MailAddress(_emailSettings.SenderEmail,
_emailSettings.SenderName),
            Subject = subject,
            Body = body,
            IsBodyHtml = true,
        };

        mailMessage.To.Add(toEmail);
        await smtpClient.SendMailAsync(mailMessage);
    }
}
```

Three email types are supported:

- `SendOrderConfirmationEmailAsync` — triggered after successful order placement
- `SendPasswordResetEmailAsync` — triggered by "Forgot Password" flow
- `SendRegistrationConfirmationEmailAsync` — triggered after account creation

Each method embeds its own HTML template with inline CSS for maximum email-client compatibility. SMTP credentials are stored in `appsettings.json` under `EmailSettings` and should be Gmail App Passwords (not the primary account password).

Email sending intentionally **does not block the checkout flow** — the order confirmation email is sent in a try/catch after the transaction commits, and failures are logged as warnings rather than errors. The checkout succeeds regardless of email delivery status.

5.11 Analytics Service

The `AnalyticsService` provides server-side aggregated data for the admin dashboard:

```
public async Task<SalesAnalyticsViewModel> GetSalesAnalyticsAsync()
{
    var model = new SalesAnalyticsViewModel();
    var orderQuery = _unitOfWork.Orders.GetQueryable(asNoTracking: true);
    var now = DateTime.Now;
    var thisMonthStart = new DateTime(now.Year, now.Month, 1);

    // Overview
    model.TotalOrders = await orderQuery.CountAsync();
    model.TotalRevenue = await orderQuery.SumAsync(o => o.TotalAmount);
    model.AverageOrderValue = model.TotalOrders > 0 ? model.TotalRevenue /
model.TotalOrders : 0;

    // Month-over-month growth
    model.OrdersThisMonth = await orderQuery.CountAsync(o => o.OrderDate >=
thisMonthStart);
    model.RevenueThisMonth = await orderQuery.Where(o => o.OrderDate >=
thisMonthStart)
        .SumAsync(o => o.TotalAmount);
```

```
    // Top selling products (aggregated server-side)
    var topProducts = await
_unitOfWork.OrderItems.GetQueryable(asNoTracking: true)
        .GroupBy(oi => oi.ProductId)
        .Select(g => new { ProductId = g.Key, QuantitySold = g.Sum(oi =>
oi.Quantity) })
        .OrderByDescending(x => x.QuantitySold)
        .Take(10)
        .ToListAsync();

    // Daily sales (last 30 days) with zero-fill for days with no orders
    model.DailySales = await GetDailySalesAsync(30);

    return model;
}
```

All aggregation happens at the database level (LINQ translates to SQL `GROUP BY`, `SUM`, `COUNT`), avoiding in-memory loading of entire tables. The zero-fill pattern for daily sales ensures chart rendering does not have gaps for inactive days.

5.12 Dependency Injection Summary (Program.cs)

Lifetime	Service	Implementation
Scoped	`IUnitOfWork`	`UnitOfWork`
Scoped	`IGeminiService`	`GeminiService`
Scoped	`IPaymentService`	`StripePaymentService`
Scoped	`IImageService`	`ImageService`
Scoped	`IEmailService`	`EmailService`
Scoped	`IAnalyticsService`	`AnalyticsService`
Singleton	`IMemoryCache`	(built-in)
Singleton	`IConfiguration`	(built-in)
Singleton	`IHttpClientFactory`	(built-in, via `AddHttpClient`)
Scoped	`UserManager<ApplicationUser>`	(Identity)
Scoped	`RoleManager<IdentityRole>`	(Identity)
Scoped	`SignInManager<ApplicationUser>`	(Identity)
Scoped	`ApplicationDbContext`	(EF Core)

All services are **scoped** (one instance per HTTP request), matching the `DbContext` lifetime. Singletons are used only for truly stateless or shared services (cache, configuration).